VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

MINI PROJECT REPORT
on

Reinforcement learning in Mixed Traffic Environment

Submitted by

S Gaja Lakshmi (1BM23CS278)
S Nafees (1BM23CS279)
Saanvi S (1BM23CS282)
Sachit P Naidu (1BM24CS420)

Under the Guidance of
Dr. Pallavi G B
Professor, B.M.S.C.E.

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B. M. S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019

August 2025 to December 2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project work entitled "**Reinforcement learning in Mixed Traffic Environment**" carried out by **S Gaja Lakshmi (1BM23CS278), S Nafees (1BM23CS279), Saanvi S (1BM23CS282), Sachit P Naidu (1BM24CS420),** who are bonafide students of **B. M. S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visveswaraiah Technological University, Belgaum during the year 2023-2024. The project report has been approved as it satisfies the academic requirements in respect of **Mini Project (23CS5PWMIP)** work prescribed for the said degree.

Signature of the Guide
Dr. Pallavi G B
Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Kavitha Sooda
Professor & Head, Dept. of CSE
BMSCE, Bengaluru

External Viva

Name of the Examiner | Signature with date

1._____ _____

2. _____ _____

B. M. S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECALARATION

We, S Gaja Lakshmi (1BM23CS278), S Nafees (1BM23CS279), Saanvi S (1BM23CS282), Sachit P Naidu(1BM24CS420), students of 5th Semester, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, here by declare that, this Project Work-entitled "Reinforcement Learning in Mixed Traffic Signals" has been carried out by us under the guidance of Dr. Pallavi G B, Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester August 2025 - December 2025.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

S Gaja Lakshmi (1BM23CS278)

S Nafees (1BM23CS279)

Saanvi S (1BM23CS282)

Sachit P Naidu (1BM24CS420)

1. Introduction

1.1

Urban traffic congestion continues to be a problem, mainly due to the fact that the conventional pretimed and actuated traffic signal control systems are unable to adapt quickly to the changing traffic situation. To this end, an innovative approach to managing traffic depending on real-time data was via Reinforcement Learning (RL), where a traffic signal agent is capable of learning the different phases of traffic by himself. This study advocates for the Q-Learning and Deep Learning of Q-Networks, with an intelligent traffic light agent observing the traffic state and suggesting actions aimed at the reduction of delays. Unlike the traditional traffic rules, the RL-based traffic light strategies are capable to change according to the traffic pattern fluctuations and asymmetries. The results of the simulations indicate that significant reductions in the numbers of cars waiting, the time spent waiting, and the overall traffic congestion have taken place. The project goes interactive and even presents a real-world application case of designing, building, and testing based on active traffic light control systems that use RL. Simulation experiments have shown that the ML-powered control schemes outperform the conventional ones in terms of queue sizes, average waiting times, and total intersection clearance throughput.

1.2 Motivation

Traffic congestion causes serious problems such as travel time increase, fuel wastage, and increased emissions. The conventional traffic signal control systems lack dynamic adaptation, especially during unpredictable traffic flow conditions like peak hours, special events, and mixed traffic environments. The motivation behind this project:

- **Adaptive control:** There is great potential for reductions in delay with real-time adjustments to actual traffic flow.
- **Limitations of fixed or rule-based controllers:** They do not learn from past performance and cannot respond to complex traffic patterns.
- **RL and computing advances:** With new versions, it's now possible to deploy RL algorithms like Q-Learning and DQN efficiently in simulation environments such as SUMO.
- **Real-life applicability:** In fact, most cities around the world are already moving towards ITS, which clearly implies the need for exploring AI-driven control strategies.

- **Ease of Value-Based RL:** Q-Learning and DQN are simpler to implement and analyze as compared to other policy-gradient algorithms; this makes them very suitable for educational purposes and research prototypes.

1.3 Scope of the Project

The project focuses on applying reinforcement learning to a **single intersection** using Q-Learning and Deep Q-Learning.

- Formulation of the traffic signal control task as a **Markov Decision Process (MDP)**.
- Design of :
  - **State representation** (e.g., queue lengths, lane occupancy).
  - **Action space** (phase switching decisions).
  - **Reward function** (delay reduction, waiting time minimization).
- Implementation of:
  - **Q-Learning agent** (tabular).
  - **Deep Q-Learning agent** (neural network for value approximation).
- Simulation using **SUMO + TraCI** to replicate realistic traffic flow.
- Comparison with:
  - Fixed-time controller.
  - Simple actuated controller

1.4 Problem statement

Traditional traffic signal controllers are limited in their ability to adapt to changing traffic conditions, resulting in increased delays and congestion. The problem addressed in this project is:

"How can reinforcement learning (Q-Learning and Deep Q-Learning) be applied to traffic signal control to dynamically optimize signal phases in order to minimize traffic congestion at an intersection?

2. Literature Survey

1. **Enhancing Robustness of Deep Reinforcement Learning Based Adaptive Traffic Signal Controllers in Mixed Traffic Environments Through Data Fusion and Multi-Discrete Actions**

   It proposes a PPO-based traffic controller that fuses CV and camera data, using a multi-discrete action space to jointly choose phases and timings. Results have shown major delay reduction even at low CV penetration, outperforming DQN and actuated control. It highlights the importance of multimodal inputs and richer action design for reliable signal control.

2. **Cooperative Control Strategies for Connected Automated Vehicles and Adaptive Traffic Light Systems in Mixed Traffic Intersections**

   This work proposes a cooperative algorithm that coordinates CAVs and HDVs with adaptive traffic lights through mixed platoons using the MP-iDFST scheduling method. The simulation results reflect substantial gains in delay, evacuation time, and fuel use. It thus shows how integrating vehicle behaviour with signal logic can optimize mixed-traffic intersections.

3. **Strategic Traffic Management in Mixed Traffic Road Networks: A Methodological Approach Integrating Game Theory, Bilevel Optimization, and C-ITS**

   This paper models mixed traffic within a multiclass assignment framework, incorporating UE, SO, and CN driver behaviours. It investigates, via game-theoretic formulations, how the presence of different driver types affects the optimality of traffic conditions. It provides insights into system-level traffic policies for heterogeneous road users.

4. **Adaptive Traffic Light Management for Mobility and Accessibility in Smart Cities.**

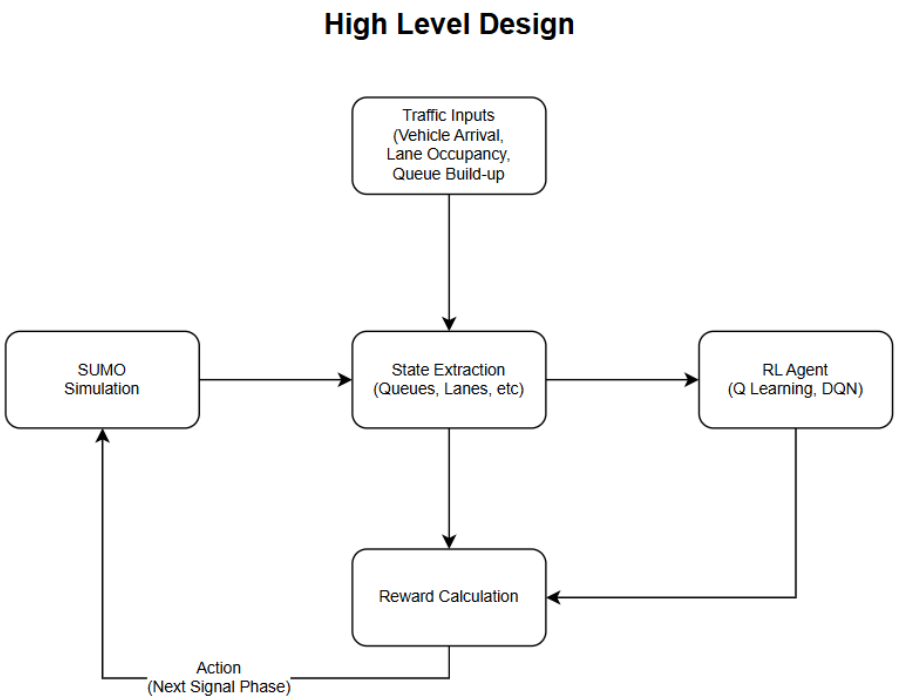
   This paper proposes an RL-based DDPG approach (H-ATLM), in which real-time sensor data is used to dynamically adjust the signal timings. Various experiments demonstrate reductions in congestion and improvements in throughput. The continuous-action RL is picked out here to be a strong option for smart-city traffic optimization.

5. **Mixed-Traffic Deep Reinforcement Learning and Multi-Source Data Approaches (General Literature Groups)**

   Recent works point to the challenges of DRL under mixed traffic, such as low CV penetration, noisy perception, and unstable learning. Most works propose combining sensor data or CV information, or even with advanced RL algorithms, such as PPO or DDPG, for more robust signal control. They all collectively underline the need to have a reliable state estimation and adaptive decision-making.

3. Design

3.1 High Level Design



The high-level design provides the overall architecture of the reinforcement learning-based traffic signal control system and depicts how the different components interact during a learning and control process.

- Traffic Inputs: The topmost portion of the system, showing the intersection in real time with vehicle arrival, lane occupancy, and queue build-up. In each case, these raw parameters of traffic are generated either inside SUMO or via simulated sensors. The information feeds into the State Extraction module, where the raw data gets processed into a structured state format that the RL agent can understand—such as queue lengths, current lane occupancy, or traffic density on each approach.
- The SUMO Simulation provides the environment in which traffic movement, vehicle dynamics, and signal behaviour are modelled. SUMO continuously feeds updated traffic conditions into the State Extraction block and implements any signal-phase changes commanded by the RL agent.

- The processed state is then fed into the RL Agent, Q-Learning/DQN, which decides upon an action to take on that given intersection state. Based on its learned policy, an action (commonly the next traffic signal phase or whether to switch or hold the current phase) is chosen by the agent. Chosen actions are then forwarded back to the SUMO environment, at which point the traffic signal will be updated according to those actions.
- It calculates the numerical reward by considering the metrics such as reduction of the queue, improvement in the waiting time, or reduction of delay. This module would send the reward to the RL Agent so that it can learn the best action to cause better traffic flow. The feedback is always on because SUMO will again and again update the traffic conditions w.r.t. the chosen actions, creating the core RL cycle - state → action → reward → learning.

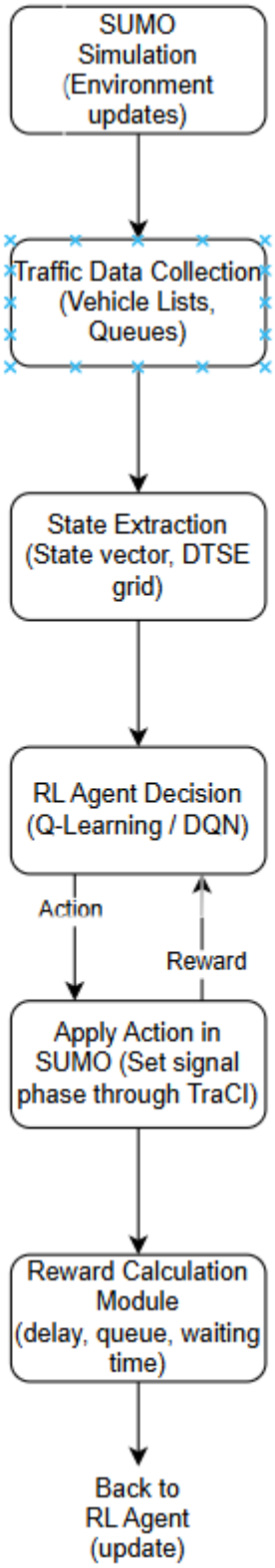
The architecture is such that the RL controller will continuously interact with the traffic environment, learn from its performance, and gradually improve upon decision-making for the minimum congestion of traffic and thereby smooth the flow.

3.2 Detailed Design.

The detailed design explains how the internal workflow of a reinforcement learning-based traffic signal controller works. This starts with SUMO Simulation, which proceeds with constant updates concerning vehicle movements and traffic conditions in a continuous manner at the intersection. These updates are then extracted in the Traffic Data Collection module in a raw form using TraCI.

This data is then processed in the State Extraction module, where it is formatted into a structured form of representation like a feature vector and a DTSE grid. This extracted state information is then delivered to the RL Agent, which adopts a choice concerning the next traffic signal changeover using either Q-Learning or Deep Q-Networks. The resultant action, usually a transition or holding of a traffic signal phase, is then implemented in the Apply Action in SUMO module using TraCI.

After performing actions, there is the evaluation of the impact of the decision taking into consideration factors such as delay reduction, queue size, and waiting time, which becomes a reward. This reward is then used to train the RL Agent. This continuous loop allows the RL Agent to learn optimal traffic signal control policies.


SUMO Simulation (Environment updates)
Traffic Data Collection (Vehicle Lists, Queues)
State Extraction (State vector, DTSE grid)
RL Agent Decision (Q-Learning / DQN)
Action
Reward
Apply Action in SUMO (Set signal phase through TraCI)
Reward Calculation Module (delay, queue, waiting time)
Back to RL Agent (update)

4. Implementation

4.1 Proposed methodology

• The methodology that is being described combines the traffic simulation environment of SUMO with a Q-Learning/DQN agent that will experiment in the repeated interaction to learn the optimal strategy for switching signals.

• Initially, the system is built upon a SUMO intersection network then linked to it via the TraCI interface. For each simulation step, the traffic data (vehicle lists, queue lengths, and lane occupancies) is collected from SUMO. The data is then transformed into a structured state representation which serves as input to the RL agent. Depending on the model, either the Q-table gets updated (Q-Learning) or the state has been fed into a neural network to estimate Q-values for DQN.

• Then the agent using the ε-greedy strategy selects an action that either chooses the best-known action or a random exploratory one. The chosen action is again applied back to SUMO by the updating the traffic signal phase through the TraCI. After the action is executed, the system computes a reward reflecting the traffic performance metrics, which can be a reduction in queue length, waiting time, or delay. This reward is used to update the agent's policy that might lead to gradual improvement in decision-making.

• This is circular reasoning: state extraction, action selection, action execution, and reward calculation. This loop continues about for tens of steps and episodes until the agent reaches an effective signal control strategy. The entire methodology permits the traffic signal controller to learn from real-time traffic behavior adaptively, rather than relying on fixed-time or rule-based control.

4.2 Algorithm used for implementation

Algorithm: Emergency Vehicle Priority Preemption System

Input: SUMO simulation, vehicle positions, edge IDs, TLS phase information

Output: Traffic signal phases modified to give priority to emergency vehicles

Steps :

1. **Initialize Environment**
   - Load SUMO configuration and start TraCI connection.
   - Define mapping between each traffic light, vehicle direction (NS/EW), and the phase that should turn green for emergency vehicles.
   - Create a dictionary to track which traffic lights have been temporarily modified.
2. **Simulation Loop**, Repeat until no vehicles remain in the network:
   1. Advance the SUMO simulation by one step.
   2. Execute the emergency vehicle handling procedure.
3. **Identify Emergency Vehicles**
   - Retrieve the list of all vehicles in the simulation.
   - Filter vehicles whose type is "emergency".
4. **Determine Vehicle Direction**, For each emergency vehicle:
   - Read its current road ID (edge).
   - If the edge name contains “nb/sb”, classify as **North–South (NS)**.
   - If the edge name contains “eb/wb”, classify as **East–West (EW)**.
5. **Locate the Next Traffic Light**
   - Use vehicle.getNextTLS() to find the next traffic signal the emergency vehicle will reach.
   - Extract: TLS ID, distance, and current signal phase.
6. **Select Required Phase for Priority**
   - Construct a key using <TLSID>_<direction>.
   - Look up the desired phase index from the predefined phase-mapping dictionary.
7. **Apply Priority Preemption**

   If the traffic light is not already in the correct phase:

- **Case A: Current phase is already the desired phase**
  - Extend the green phase duration (e.g., +10 seconds, minimum 20 seconds).
- **Case B: Current phase is different**
  - Shorten the current phase duration to a very small value (e.g., 0.1 sec) to quickly switch to the emergency phase.
- Mark this traffic light as "adjusted" so it is not modified repeatedly.

8. **Track Active Priority Signals**
   - Keep a set of all TLS IDs that currently have an emergency vehicle approaching.
9. **Reset Normal Operation**
   - For each traffic light previously adjusted:
     - If no emergency vehicle is currently heading towards it, remove it from the adjusted list and allow SUMO to resume normal cycle timing.
10. **Close Simulation**
    - After all vehicles are cleared, stop TraCI and end the program.

Algorithm: Q-Learning–Based Traffic Signal Control

Input: Detector readings, SUMO traffic state, current signal phase
Output: Optimized traffic signal actions (keep phase / switch phase)

1. Initialization

1. Import required libraries (SUMO, TraCI, numpy, random).
2. Load SUMO configuration and start the TraCI connection.
3. Initialize:
   - Queue variables
   - Learning parameters (α, γ, ε)
   - Action set **A = {0 = keep phase, 1 = switch phase}**
   - Q-table as an empty dictionary
   - Minimum green time counter
   - Step counters and logging lists

2. State Extraction

At every simulation step:

1. Read queue lengths from detectors:
   - EB_0, EB_1, EB_2
   - SB_0, SB_1, SB_2
2. Read the current traffic signal phase.
3. Construct the state tuple:

$$s = (q_{EB0}, q_{EB1}, q_{EB2}, q_{SB0}, q_{SB1}, q_{SB2}, \text{phase})$$

3. Action Selection (ε-greedy policy)

For the current state s:

1. With probability **ε** → choose a random action (exploration).
2. With probability **1 − ε** → choose action with maximum Q-value:

$$a = \arg\max_{a'} Q(s, a')$$

If state does not exist in Q-table, initialize Q(s) = [0, 0].

3. **Apply Action**
   1. If action = **0** → do nothing (keep current phase).
   2. If action = **1** →
      1. Check if the minimum green time has elapsed: If allowed switch to next phase and update last_switch_step.

$$\text{step} - \text{last_switch_step} \geq \text{MIN_GREEN_STEPS}$$

5. Environment Step

1. Advance the SUMO simulation by 1 step.
2. Observe the new state s'after the action.

6. Reward Calculation
Compute reward as:

$$r = -(\text{sum of all queue lengths})$$

A shorter queue → higher reward (less negative).

7. Q-Value Update

For old state s, action a, reward r, and new state s':

1. Ensure Q(s') exists in table
2. Compute the target value:

$$\text{target} = r + \gamma \max_{a'} Q(s', a')$$

3. Update using Q-learning rule:

$$Q(s, a) = Q(s, a) + \alpha(\text{target} - Q(s, a))$$

8. Logging

Every training step, store:

- current step
- cumulative reward
- total queue length
- updated Q-values

Used for plotting and analysis.

9. Loop Until Training Ends

Repeat steps **2–8** for **TOTAL_STEPS**.

When finished:

- Print final Q-table
- Plot cumulative reward vs steps
- Plot queue length vs steps

Algorithm: Deep Q-Learning (DQN) for Traffic Signal Control

Input:

Queue lengths from detectors, current phase, SUMO traffic environment

Output:

Optimized traffic signal action (keep phase / switch phase)

1. Initialization

1. Import SUMO, TraCI, TensorFlow, NumPy, and helper libraries.
2. Start SUMO using the provided configuration file.
3. Define the state variables:
    - Queue lengths from EB and SB detectors
    - Current signal phase
4. Set RL hyperparameters:
    - Learning rate α
    - Discount factor γ
    - Exploration rate ε
    - Action set **A = {0 = keep phase, 1 = switch phase}**
5. Build a **neural network** to approximate Q-values:
    - Input layer (state size = 7)
    - Dense(24, ReLU)
    - Dense(24, ReLU)
    - Output layer = number of actions
6. Initialize minimum green-time constraint tracker.

2. State Extraction

At every simulation step:

1. Read queue lengths from lane-area detectors (EB_0, EB_1, EB_2, SB_0, SB_1, SB_2).
2. Read the current traffic signal phase from SUMO.
3. Construct the state:

$$s = (q_{EB0}, q_{EB1}, q_{EB2}, q_{SB0}, q_{SB1}, q_{SB2}, \text{phase})$$

3. Action Selection (ε-greedy using DQN model)

For current state s:

1. With probability **ε** → choose a random action.
2. Otherwise → use the DQN model to predict Q-values:

$$a = \arg\max Q(s)$$

4. Action Execution

1. If action = **0** → keep the current signal phase unchanged.
2. If action = **1** →
   - Check if **MIN_GREEN_STEPS** have passed since the last phase change.
   - If yes, compute the next phase and apply it using TraCI.
   - Update **last_switch_step**.

5. Environment Step

1. Advance SUMO by one simulation step.
2. Observe the new state s'.

6. Reward Calculation

Reward is defined as the negative sum of all queue lengths:

$$r = -\sum(\text{queue lengths})$$

Less queue → higher reward.

7. DQN Update (Single-Step Gradient Update)

1. Predict Q-values for the old state s.
2. Predict Q-values for the new state s'.
3. Compute the TD target:

$$\text{target} = r + \gamma \max Q(s')$$

4. Update only the Q-value of the taken action:

$$Q(s)[a] \leftarrow Q(s)[a] + \alpha(\text{target} - Q(s)[a])$$

5. Train the neural network on this updated target using one gradient step.

8. Logging and Monitoring

Every step:

- Store cumulative reward
- Record queue length
- Print network Q-values for debugging

Graphs are plotted for:

- Cumulative reward vs steps
- Total queue length vs steps

9. Termination

After all training steps:

- Close TraCI connection
- Display DQN model summary
- Plot final performance graphs

4.3 Tools and technologies used

1. **SUMO (Simulation of Urban Mobility)**

   Used as the main traffic simulator to model vehicle movements, intersections, and traffic lights.

2. **TraCI (Traffic Control Interface)**

   A Python interface used to control SUMO in real time, read traffic data, and apply signal actions.

3. **NetEdit**

   Used to design and edit the road network, traffic signals, detectors, and routes.

4. **Python**

   The primary programming language used to implement Q-Learning and Deep Q-Learning.

5. **TensorFlow / Keras**

   Used to build and train the neural network for the Deep Q-Learning model.

4.4 Testing

Testing Scenarios.

The RL models were tested under multiple traffic conditions:

- **Normal Traffic Flow:** Regular vehicle arrivals to verify that the controller adapts to typical day-to-day traffic.
- **High Congestion Scenario:** Increased vehicle density to test the agent's ability to handle long queues and peak-hour traffic.
- **Balanced vs. Unbalanced Traffic:** Different traffic loads on East-West and North-South lanes to evaluate if the controller prioritizes the heavier direction.
- **Emergency Preemption Active:** Testing that the emergency vehicle priority system works even when RL control is running.

Testing Procedure

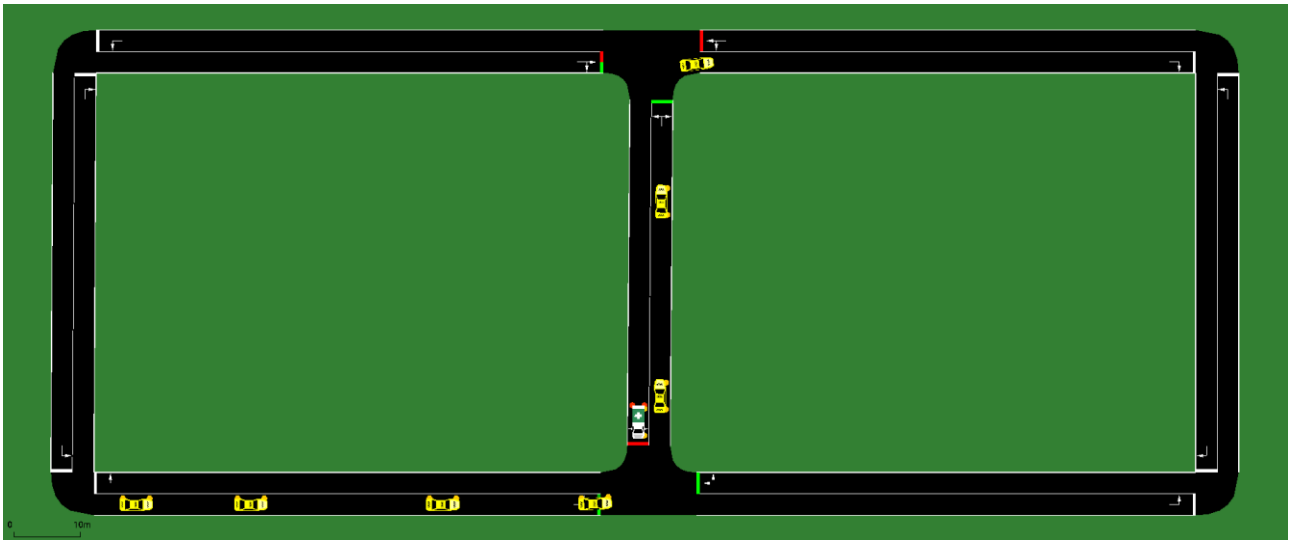
1. Initialize SUMO simulation with the defined network, detectors, and TLS phases.
2. Run the RL agent (Q-Learning or DQN) continuously for a fixed number of steps (e.g., 10,000).
3. At each step:
    - Collect queue lengths
    - Record rewards
    - Observe traffic light decisions
4. Plot performance graphs, such as:
    - Total queue length over time
    - Cumulative reward over time
5. Compare RL performance with:
    - Default fixed-time controller
    - Basic actuated controller (if available)
6. Test stability by running multiple trials.

Metrics Used for Testing

- **Total Queue Length:** Primary metric to check if RL reduces congestion.
- **Cumulative Reward:** Indicates whether the agent gradually improves traffic performance.
- **Phase Switching Behaviour:** Verifies that the agent respects minimum green times and does not oscillate rapidly.
- **Vehicle Delay (Optional):** Shows time vehicles spend waiting at the signal.

5. Results and Discussion

Without Priority Preemption: [Fig 5.1]



Without Priority Preemption (Fig. 5.1), In the baseline case, the traffic signal continuously shifts under a fixed-time standard or actuated logic without being aware of the emergency vehicles.

As depicted in Fig. 5.1, the emergency vehicle is held up behind the usual traffic and is creeping through the network.

The absence of priority leads to:

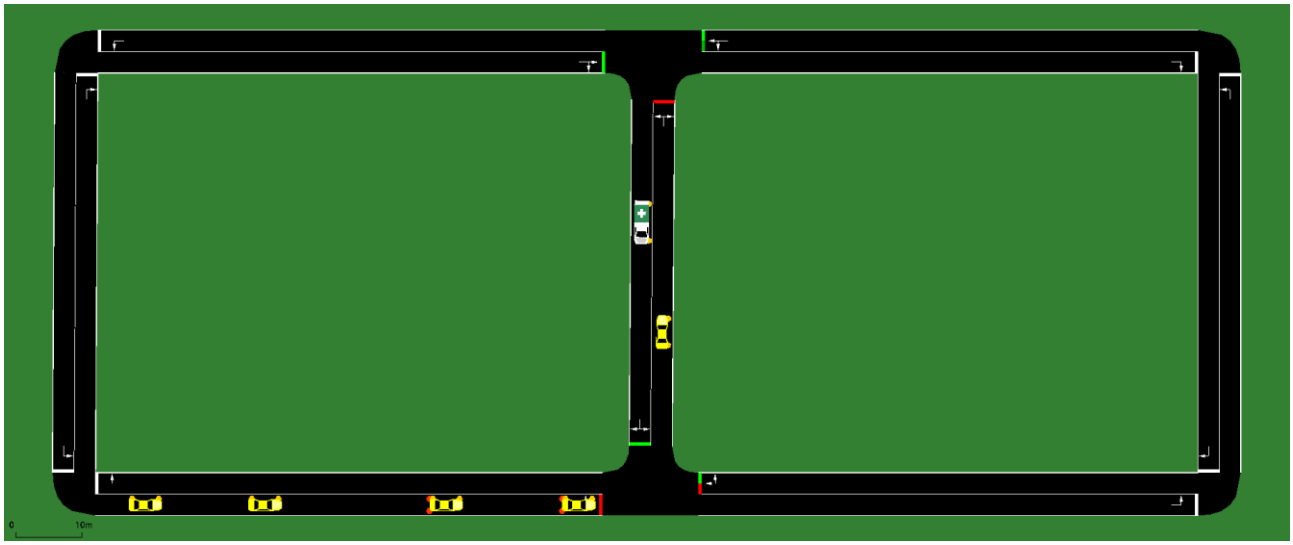
Waiting for the emergency vehicle for a longer time

Overlapping of congestion as following traffic starts to queue

Decreased throughput of the whole network, as the front lanes are held up longer

Since the controller does not differentiate between vehicles, the emergency one gets the red light as well, which results in longer and slower legal crossing of the intersection, thereby, risking the response time and safety.

With Priority Preemption: [Fig 5.2]



With Priority Preemption (Fig. 5.2) If the priority preemption is on, the traffic signal will recognize the emergency vehicle and will change the signal phases in a way that there will be a clear road for the vehicle.

In Fig. 5.2, the emergency vehicle is given a green wave, so it can go through the intersection without stopping.

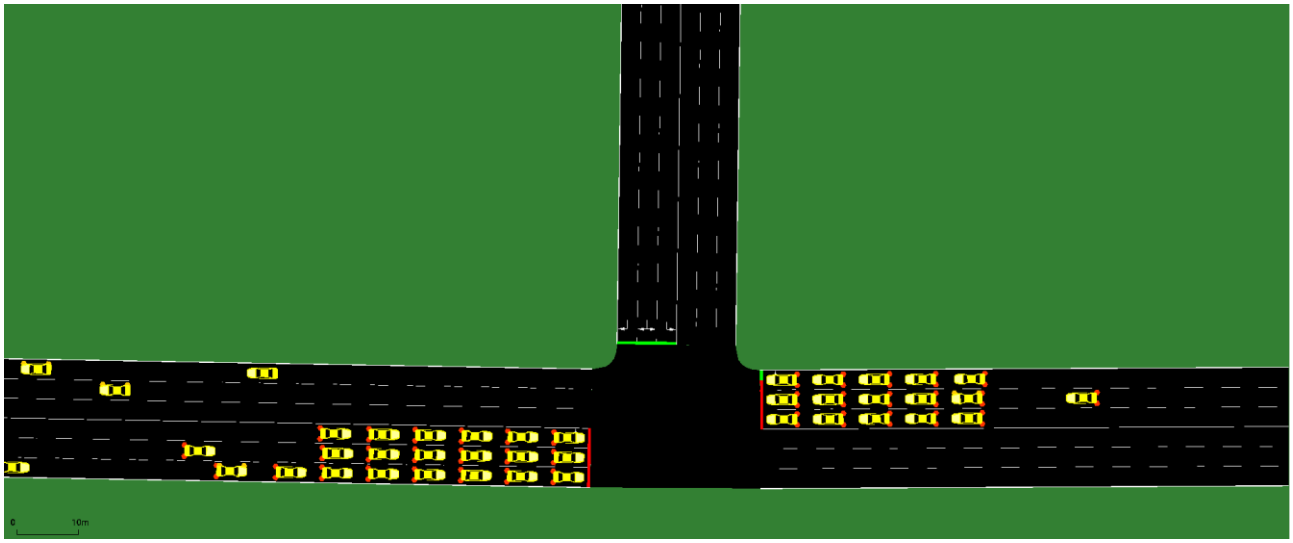
This brings about:
Greatly reduced emergency response time

Downstream lanes soon cleared and no queue built up

Least disruption to the regular traffic as the preemption works only when it is needed

Overall flow stability improved because there is no building up of congestion behind the emergency vehicle.
The comparison vividly shows that priority preemption facilitates mobility, safety, and operational efficiency in emergency situations.

Without Reinforcement Algorithm (Time based Traffic signals) [Fig 5.3]



Without Reinforcement Algorithm (Fixed-Time Signals) — Fig. 5.3 Under the typical fixed-time control method, traffic signals follow an unchanging, predetermined cycle that does not synchronize with the instantaneous traffic demand.

Such a scenario illustrated in Fig. 5.3 leads to:

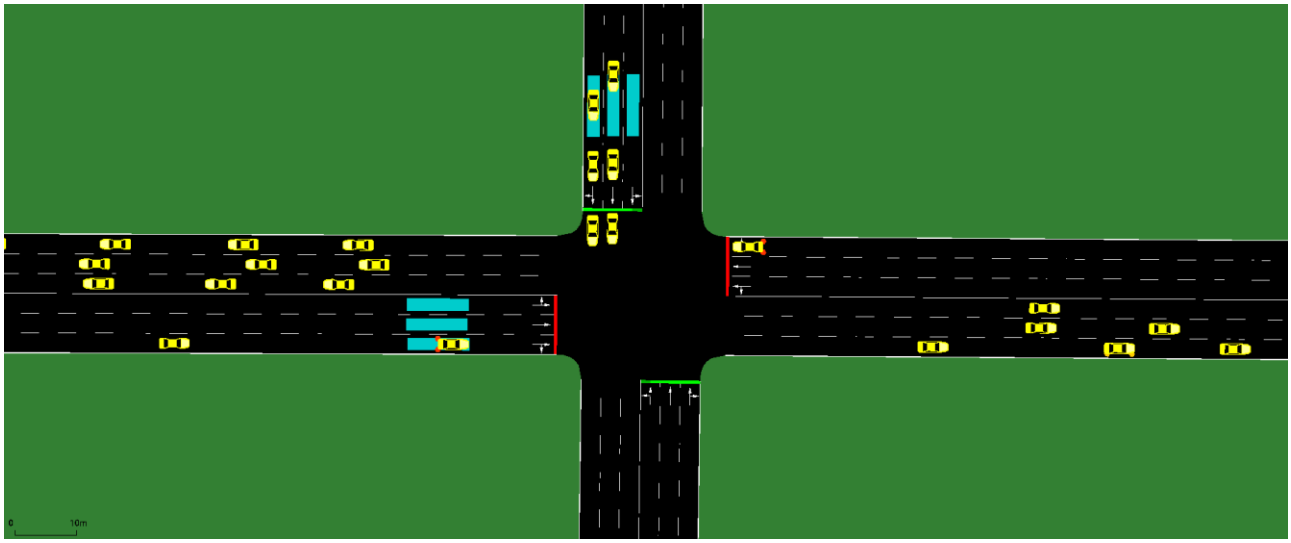
Queue formation to a large extent, especially on roads that are heavily trafficked. Inefficient use of green time when some lanes are waiting while others are overflowing with vehicles.

Bigger waiting time and delay because of the controller's incapacity to handle sudden traffic volume increases.

Traffic distribution that is unbalanced, as already mentioned, some movements quickly clear while others still are congested.

The fixed-time controller at every phase gives the same treatment without considering vehicle density, which results in the formation of queues and unstable traffic conditions during peak flow.

With Reinforcement Algorithm [Fig 5.4]



With Reinforcement Algorithm — Fig. 5.4 In case of reinforcement learning (RL) being applied to the intersection, the controller with the signal phases modifies dynamically according to the length of the queues and the state of traffic at that moment.

In Fig. 5.4, several benefits are shown:

There are completely reduced queues of vehicles for all approaches because of the adaptive timing.

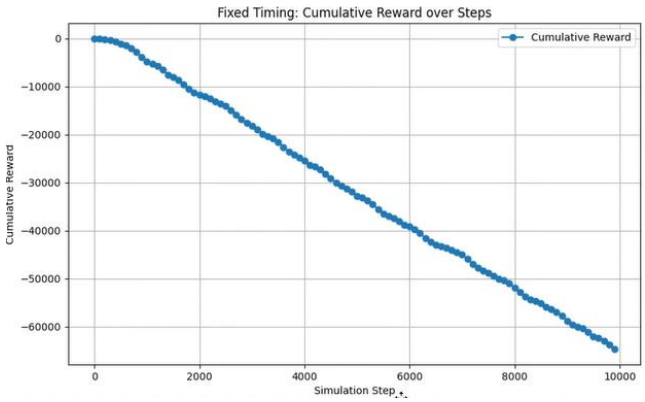
The lane with heavy traffic is cleared faster because the RL agent gives green time more where it is required.

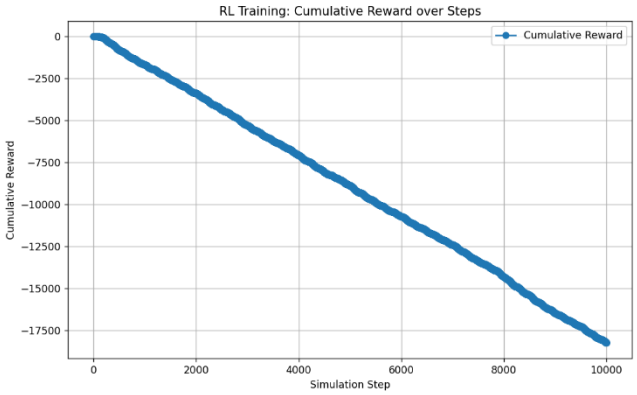
Lower total delay and waiting time, as the system is reacting to real-time traffic conditions.

The intersection performance is more balanced and thus, one direction will not dominate or block the junction.

The RL agent becomes proficient in making the right switches by learning from continuously monitoring queue lengths and getting feedback (reward), resulting in a traffic light strategy that is more effective than static timing plans.

Comparison between Reinforcement and Time based Signals





The two charts illustrate the growth of cumulative reward over 10,000 simulation steps:

• Left chart: Fixed-time signal control

• Right chart: Reinforcement learning–based control

As per our reward formulation: Reward = – (total queue length) and the penalty for long queues is reflected as a more negative cumulative reward, thus worse performance is implied.

1. Fixed Time Traffic Signals

• There is a very steep drop in the cumulative reward to around –60,000 by the end of the 10,000th step.

• The negative slope points consistently high queue lengths all through the simulation.

• The process is irregular with a little rough drops, indicating that fixed-time logic often creates sudden big queues when demand does not correspond with signal timing. Fixed-time control is incapable of adjusting to the varying traffic conditions. Thus:

• The heavy traffic lanes remain congested for long durations

• The green light durations are wasted on the lanes with no or little traffic

• The car queues keep on increasing, leading to the accumulation of large negative rewards

2. Reinforcement Learning Based Traffic Signals

• The cumulative reward drops very slightly in comparison to the case of fixed-time signals.

• By the 10,000th step, the cumulative reward has reduced to about –18,000, which is a much less negative number compared to the fixed-time case.

• The appearance of the graph is that of smoothness, implying the queue patterns were more stable and did not have extreme congestion spikes as in the case of fixed-time signals. Reinforcement Learning is in fact changing the signal timings according to the real-time traffic queues.

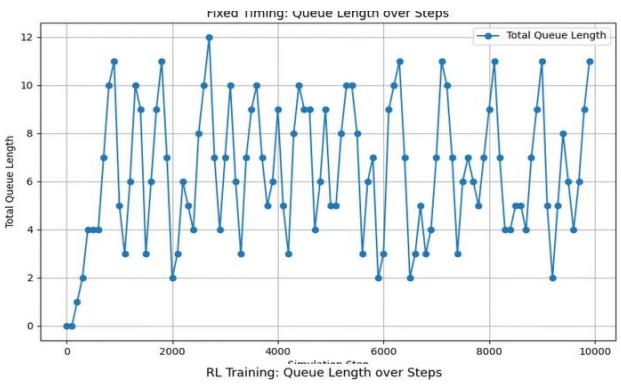
This leads to:

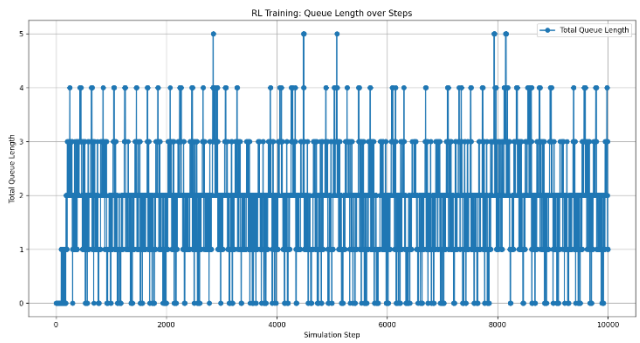
• Reduction in the size of queues

• Increased efficiency at the intersection

• Better selection of phases

• Faster dispersal of traffic in the case of congested approaches





6. Conclusion and Future Work

The effectiveness of reinforcement learning—namely Q-Learning and Deep Q-Learning—was shown in this project for adaptive traffic signal control in a simulation environment of SUMO. The system, by letting the agent learn directly from real-time queue data and vehicle behavior, achieved smoother traffic flow, shorter queues, and better intersection performance in essence over fixed-time control. Compared to the tabular Q-Learning approach, the neural network Deep Q-Learning model, which approximates Q-values, exhibited better adaptability to dynamic traffic conditions as well as quicker learning. Moreover, an emergency vehicle preemption mechanism was effectively integrated ensuring that emergency vehicles got a clear priority path without causing major disturbances to regular traffic flow. In conclusion, the findings indicate that reinforcement learning can be a potent alternative to rule-based or scheduled signal systems. The project verifies that RL-based traffic control can shorten waiting times, increase throughput, and respond intelligently to the variations in traffic.

Future Work

While the system performed well in simulation, several avenues exist for further enhancement:

1. Real-World Deployment with IoT Integration
A major next step is to implement the system in a real-world environment using IoT hardware such as:

- roadside sensors
- Raspberry Pi/ESP32 controllers
- wireless communication modules

2. Use of Advanced Reinforcement Learning Algorithms
Future versions may replace Q-Learning or DQN with more robust RL algorithms such as:

- PPO (Proximal Policy Optimization)
- A3C / A2C

References:

[1] M. Elhenawy, A. Almannaa, W. Shao, and H. A. Rakha, "Enhancing Robustness of Deep Reinforcement Learning–Based Adaptive Traffic Signal Controllers in Mixed Traffic Environments Through Data Fusion and Multi-Discrete Actions," IEEE Transactions on Intelligent Transportation Systems, vol. 24, no. 5, pp. 5444–5456, 2023.

[2] Y. Feng and H. X. Liu, "Cooperative Control Strategies for Connected Automated Vehicles and Adaptive Traffic Light Systems in Mixed Traffic Intersections," Transportation Research Part C: Emerging Technologies, vol. 134, 2022.

[3] M. S. Ghiasi, X. Li, and Z. Qian, "Strategic Traffic Management in Mixed Traffic Road Networks: A Methodological Approach Integrating Game Theory, Bilevel Optimization, and C-ITS," Transportation Research Part C: Emerging Technologies, vol. 152, 2023.

[4] L. Rizzi, F. Babich, and M. Comisso, "Adaptive Traffic Light Management for Mobility and Accessibility in Smart Cities Using DDPG-Based Reinforcement Learning," IEEE Access, vol. 10, pp. 104948–104964, 2022.

[5] Composite summary of key mixed-traffic DRL challenges based on contemporary research trends (sensor fusion, CV penetration, and RL stability), derived from the above four primary papers.